EAST WEST UNIVERSITY

# Project Report

## Title: Tomato Disease Classification Using CNN Architectures

**Course Title: Artificial Intelligence**

**Course Code: CSE366**

**Semester: Spring 2026**

**Section: 01**

**Group: 08**


**Submitted By:**

| | |
|---|---|
| **MD. Rejaul Hoq** | **2023-1-60-124** |
| **Meghla Biswas** | **2023-1-60-225** |
| **Md. Rahat Khan** | **2021-1-60-090** |
| **Nahidul Islam** | **2022-3-60-072** |

**Submitted To:**

**Dr. Raihan Ul Islam**

**Associate Professor**

**Department of Computer Science and Engineering**


*Date of Submission: 20/04/2026*

# 1.Introduction

Plant diseases are a major threat to food security, causing substantial reductions in crop yield and quality. For crops like tomatoes, which are highly susceptible to various fungal, bacterial, and viral pathogens, timely and accurate disease detection is essential. Traditional assessment of plant health relies heavily on visual inspection by agricultural experts. While widely used, these methods can be time-consuming, expensive, and subject to observer bias or fatigue.

This has motivated a line of research into automated plant disease detection using computer vision and deep learning. Modern convolutional neural networks (CNNs) have shown exceptional capability in extracting complex hierarchical features from image data. However, many existing models operate as "black boxes" and struggle with domain shifts when applied to new environments with varying backgrounds and lighting conditions. There remains a need for methods that not only achieve high classification accuracy but also provide clear, interpretable insights into how specific visual patterns relate to a disease diagnosis.

To address these challenges, this paper introduces a comprehensive approach for classifying 10 categories of tomato leaf health (9 diseases and 1 healthy class) using both pre-trained architectures and custom-built attention-based models. The contributions of this paper are:

- We developed a robust preprocessing and augmentation pipeline tailored to leaf imagery, ensuring domain-specific learning without introducing unrealistic spatial distortions.
- We evaluated and compared multiple architectures, including ResNet50, VGG16, EfficientNetB0, and a Custom CNN enhanced with a Convolutional Block Attention Module (CBAM) to improve spatial and channel feature focus.
- We applied Grad-CAM to visualize model attention, confirming that the network decisions are driven by biologically relevant disease lesions rather than background artifacts.
- We tested the model's generalizability on a secondary dataset, demonstrating its resilience to minor domain shifts in lighting and background.

## 2.Related Work:

| Title | Dataset name | Dataset description | Method | Accuracy | Research Questions | Pros & Cons | Citation |
|---|---|---|---|---|---|---|---|
| Tomato Leaf Disease Detection using Generative Adversarial Network-based ResNet50V2 | PlantVillage Dataset https://www.kaggle.com/datasets/emmarex/plantdisease | The PlantVillage dataset containing 16,012 tomato leaf images categorized in ten classes. Among them, nine classes are tomato leaf diseases and one class are healthy leaf images with images of size 256×256 pixels | GAN-based Data Augmentation with Transfer Learning using ResNet50V2 | Accuracy: 99.75% Precision: 99.28% Recall: 99.43% F1-score: 99.67% | Can Generative Adversarial Network (GAN)- based data augmentation improve the classification accuracy and generalization of tomato leaf disease detection using deep CNN models? | Pros:<br>• Reduces overfitting using synthetic data<br>• Achieves very high classification accuracy<br>• Improves generalization performance<br><br>Cons:<br>• GAN training increases computational cost<br>• Overall training time is higher | [1] |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Tomato Leaf Disease Detection using Deep Learning Techniques | Custom Tomato Leaf Dataset (No public URL provided; laboratory-acquired dataset) | The dataset consists of 735 tomato leaf images belonging to 7 classes, including six disease categories and one healthy class. The dataset was split into 70% training and 30% testing samples. | Fuzzy Support Vector Machine (Fuzzy-SVM), Convolutional Neural Network (CNN), and Region-based Convolutional Neural Network (R-CNN) | 96.735% (R-CNN) | Which machine learning or deep learning classifier (Fuzzy-SVM, CNN, or R-CNN) provides the highest accuracy for tomato leaf disease detection? | Pros:<br>• R-CNN achieved higher accuracy compared to CNN and Fuzzy-SVM<br>• Effective feature extraction using deep learning • Suitable for early disease detection<br><br>Cons:<br>• Small dataset size<br>• High computational cost for R-CNN<br>• Dataset collected in controlled laboratory conditions only | [2] |
| ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network | PlantVillage Dataset — https://www.kaggle.com/datasets/emmarex/plantdisease | Tomato leaf images collected from PlantVillage dataset. Total 10 classes (9 disease + 1 healthy). Training | Custom Convolutional Neural Network (CNN) with 3 convolution layers, 3 max-pooling layers, and 2 fully | Average classification accuracy of 91.2% across 10 classes | Can a lightweight CNN architecture outperform pre- trained models (VGG16, Inception V3, MobileNet) for | **Pros:** High accuracy with low computational cost; fewer parameter s; suitable for mobile devices.<br><br>**Cons:** Limited to tomato leaves; | [3] |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | set: 10,000 images (1000 per class), Validation set: 7,000 images (700 per class), Test set: 500 images (50 per class). Image size: 256×256. | connected layers | | tomato leaf disease classification using PlantVillage dataset? | performance may degrade with real- field images and larger datasets. | |
| Research on Tomato Leaf Disease Classification Based on Machine Learning | PlantVillage Dataset URL: https://github.com/spMohanty/PlantVillage-Dataset | Total 10 classes, 1 healthy tomato leaf 9 tomato leaf diseases. High-resolution tomato leaf images. Dataset taken from PlantVillage. Data augmentation used (rotation, blur, noise, color | AlexNet ResNet18 YOLOv5 YOLOv8 | AlexNet: 89.25% ResNet18: 92.28% YOLOv5: 94.45% YOLOv8: 95.18% (highest accuracy) | How effectively can machine learning and deep learning models classify tomato leaf diseases?<br><br>Which deep learning model provides the best accuracy and training efficiency for tomato leaf disease classification? | **Pros:**<br>1. High classification accuracy<br>2. Comparison of multiple models<br>3. YOLOv8 shows fast training and high performance<br>4. Uses standard PlantVillage dataset<br><br>**Cons:**<br>1.Dataset has simple background images. | [4] |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | transform ) | | | | 2. Real-world complex field conditions not fully addressed<br><br>3. Limited to tomato leaves only | |
| Optimized Custom CNN for Real-Time Tomato Leaf Disease Detection | Custom Field-Collected Tomato Leaf Dataset (Bangladesh ) URL (dataset paper): https://doi.org/10.1016/j.dib.2025.111327 | Total images: 1028, 482 healthy tomato leaf images and 546 diseased tomato leaf images. Classes: 2 (Healthy, Diseased) Images captured in real field conditions | Custom CNN (proposed), YOLOv5, MobileNetV2, ResNet18 | Custom CNN: 95.2% (best) MobileNetV2: 89% YOLOv5: 84% ResNet18 : 82% | 1. Can a lightweight custom CNN outperform pre- trained models for tomato leaf disease detection? 2.How effective is a real- time, web- based deep learning system for assisting farmers in disease identification? | **Pros:**<br>1. Uses real field images, not lab- only data<br>2.Lightweight custom CNN (low storage, fast inference) 3. High accuracy with fewer parameters<br><br>**Cons:**<br>1.Only binary classification (healthy vs diseased)<br><br>2.Disease types are not separately classified | [5] |

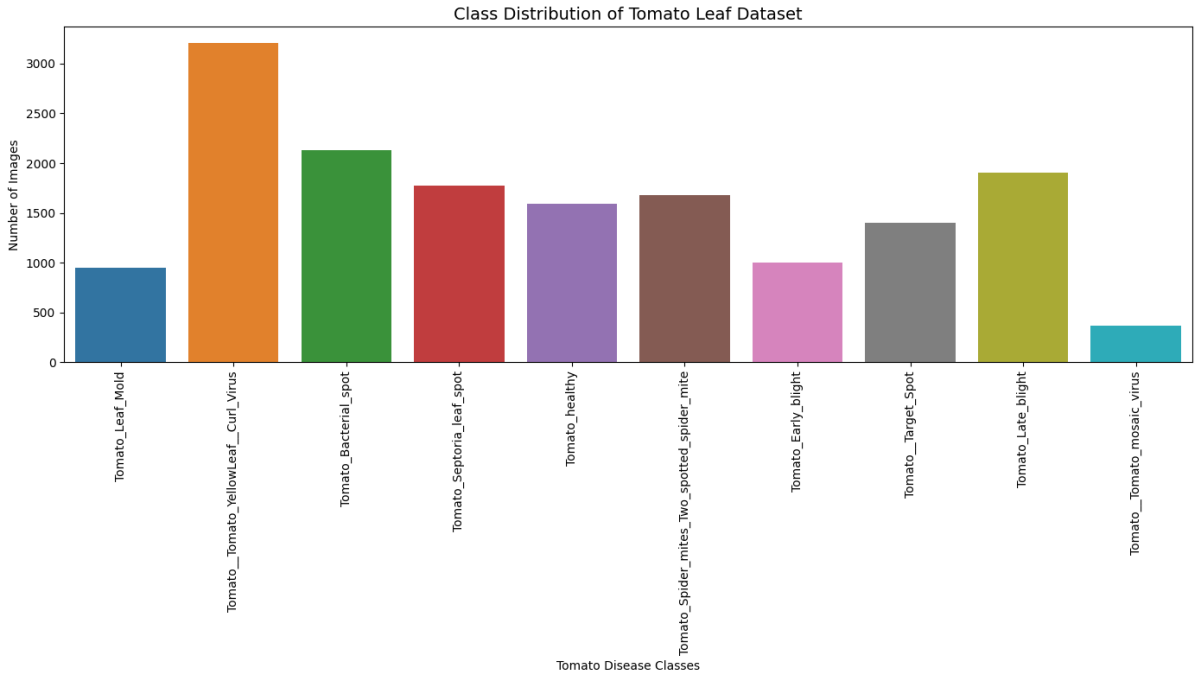
# 3.Dataset Details

The primary dataset employed in this study is the PlantVillage dataset, publicly available on Kaggle.

## 3.1. Dataset Description

The study focuses exclusively on tomato leaf images. Ten tomato disease classes were selected: Tomato_Bacterial_spot, Tomato_Early_blight, Tomato_Late_blight, Tomato_Leaf_Mold, Tomato_Septoria_leaf_spot, Tomato_Spider_mites_Two_spotted_spider_mite, Tomato__Target_Spot, Tomato__Tomato_mosaic_virus, Tomato__Tomato_YellowLeaf__Curl_Virus, and Tomato_healthy. All images were stored in RGB format and loaded using the Python Imaging Library (PIL).

Exploratory Data Analysis (EDA) revealed a moderate class imbalance across the categories.



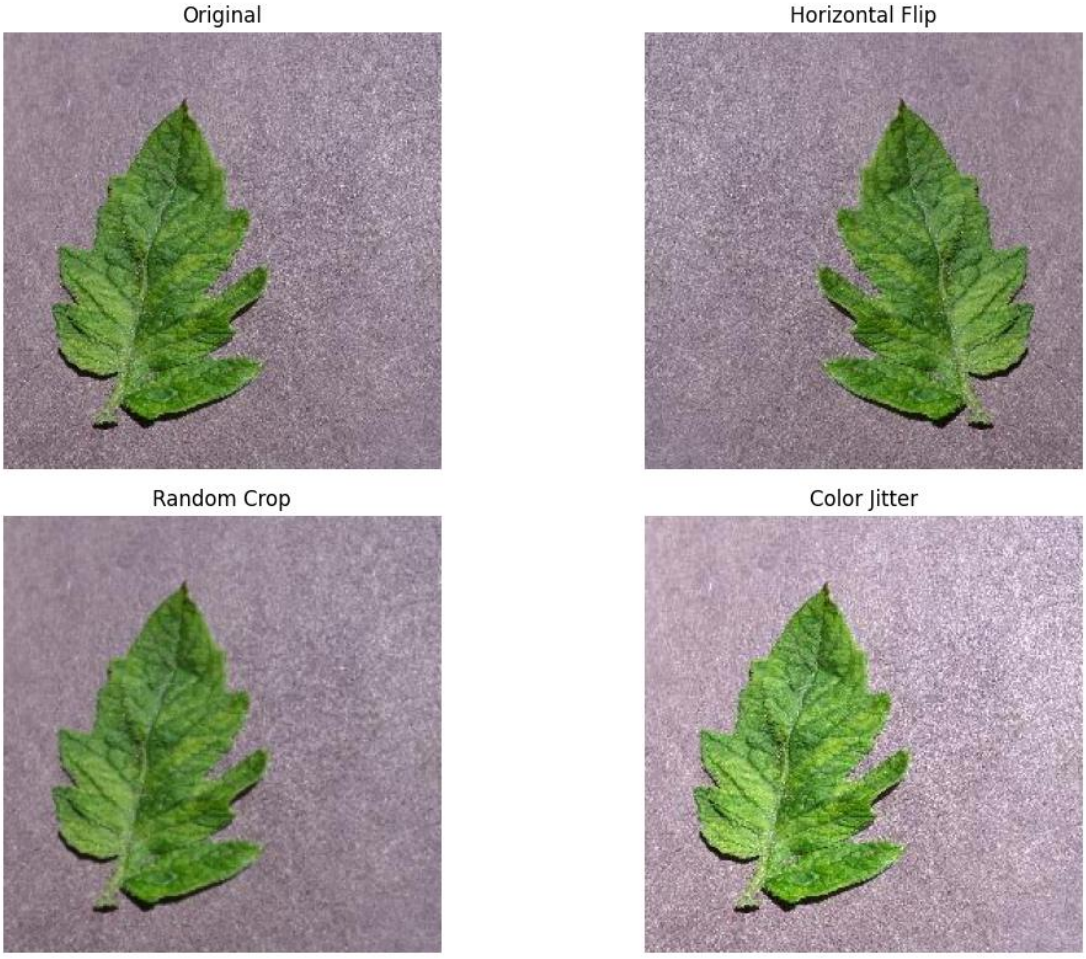

## 3.2. Image Characteristics

Sample visualization demonstrated clear intra-class consistency and strong inter-class variation. The images are generally clean with minimal background noise. Furthermore, all images are uniform in size, featuring a resolution of 256 × 256 pixels and an aspect ratio of approximately 1.0 (square), making them safe for resizing without introducing dimensional distortion. RGB channel analysis indicated a strong dominance of the green channel, reflecting the natural leaf pigmentation and confirming that the dataset is color-rich.

## 3.3. Data Augmentation

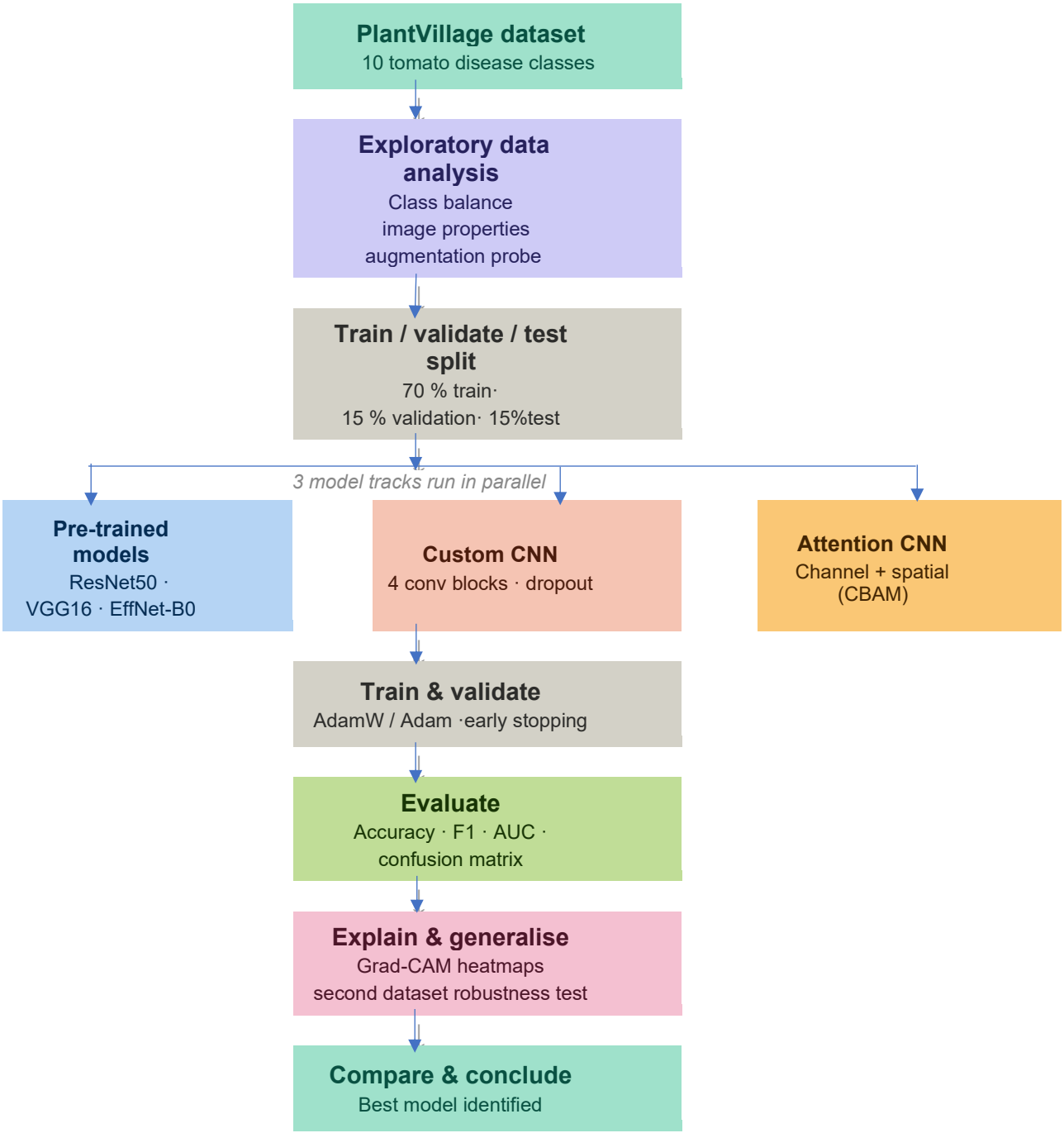
To improve model robustness and prevent overfitting, a targeted data augmentation strategy was implemented:

- **Horizontal Flip:** Preserves disease patterns (Used).
- **Mild Random Crop:** Retains crucial disease lesions (Used).
- **Color Jitter:** Simulates varying lighting conditions (Used).
- **Aggressive Rotation:** Avoided, as it introduces unrealistic leaf orientations.




## 4. Methodology

Our methodology is structured around comparing pre-trained models against custom-built and attention-enhanced architectures.




## 4.1 Exploratory Data Analysis (EDA)

Prior to model development, a comprehensive Exploratory Data Analysis (EDA) was conducted to understand the dataset characteristics and inform preprocessing strategies.

**Class Distribution Analysis**

The number of images per class was tabulated and visualized using a bar chart. This analysis revealed class imbalance, which was subsequently addressed in downstream modeling tasks through weighted loss functions. Let C = {c_1, c_2, ..., c_K} denote the set of K = 10 classes and n_k be the image count for class c_k. The imbalance ratio is defined as:

$$IR = max_k(n_k) / min_k(n_k) \quad (1)$$

**Sample Image Visualization**

For each class, three random samples were displayed in a grid layout to facilitate qualitative assessment of intra-class variation, image background complexity, and disease symptom appearance.

**Image Property Investigation**

A random sample of 50 images per class was used to measure pixel dimensions (width × height) and compute aspect ratios. Histograms were generated for width, height, and aspect ratio distributions. All images were found to share a uniform aspect ratio of 1:1, confirming that no geometric distortion would occur under square resizing.

**RGB Color Distribution Analysis**

The pixel intensity distributions across the red, green, and blue channels were plotted for a representative image. This analysis revealed that the green channel carried the dominant signal due to the leafy nature of the dataset, guiding the selection of color-based augmentation parameters.

**Augmentation Probe**

A controlled augmentation experiment was performed to evaluate the visual effect of four transformations: (i) original (no transformation), (ii) random horizontal flip, (iii) random resized crop with scale (0.8, 1.0), and (iv) color jitter with brightness and contrast factors of 0.4. This probe confirmed that horizontal flipping and random cropping preserved disease-relevant texture patterns, whereas aggressive color jitter risked distorting chlorophyll-related color signals.

## 4.2 Transfer Learning with Pre-Trained Architectures

Three pre-trained convolutional neural network architectures were fine-tuned on the tomato disease dataset: ResNet50, VGG16, and EfficientNet-B0. All models were sourced from the timm library with ImageNet pre-trained weights.

**Data Preparation**

The dataset was partitioned into a 70% training set and a 30% test set using a stratified file-copy approach. Data loading was implemented using PyTorch's ImageFolder class. Training images underwent augmentation comprising random resized cropping (scale 0.8–1.0), random horizontal flipping, and random rotation up to 20 degrees. Validation images were only resized to 224 × 224 pixels. ImageNet normalization was applied to all splits using mean $\mu = (0.485, 0.456, 0.406)$ and standard deviation $\sigma = (0.229, 0.224, 0.225)$ per channel.

**Model Architectures**

For all three architectures, the final classification head was replaced with a fully-connected layer of dimension equal to the number of target classes ($K = 10$). Specifically:

**ResNet50:** A 50-layer residual network employing skip connections to mitigate vanishing gradients. The final average-pooled feature vector was mapped to K classes.

**VGG16:** A 16-layer deep sequential network with 3×3 convolution filters and max-pooling. The three-layer fully-connected classifier was adapted for K outputs.

**EfficientNet-B0:** A compound-scaled architecture optimizing depth, width, and resolution jointly via a fixed scaling coefficient $\varphi$.

All models were trained using the AdamW optimizer with learning rate $\eta = 3 \times 10^{-4}$ and a batch size of 32.

## 4.3 Custom CNN from Scratch

A custom CNN architecture was designed and trained from scratch using TensorFlow/Keras to establish a non-pretrained baseline. The model consisted of four convolutional blocks, each comprising a Conv2D layer, Batch Normalization, ReLU activation, and MaxPooling. The final feature map was reduced via Global Average Pooling, followed by a dense layer of 128 units, dropout regularization ($p = 0.5$), and a softmax output layer.

Data preparation followed a stratified three-way split: 70% training, 15% validation, and 15% testing. The Keras ImageDataGenerator was used for on-the-fly augmentation, including random rotation (±15°), zoom (15%), width and height shifts (10%), and horizontal flipping.

To address class imbalance, inverse-frequency class weights were computed and passed to the model during training:

$$w_k = N / (K \cdot n_k) \quad (3)$$

where N is the total number of training samples, K is the number of classes, and n_k is the number of samples in class k. The model was trained for up to 50 epochs using the Adam optimizer with learning rate $\eta = 1 \times 10^{-4}$. Training was monitored using three Keras callbacks: EarlyStopping (patience = 5), ModelCheckpoint (saving the best validation-loss model), and ReduceLROnPlateau (reduction factor 0.5, patience = 2, minimum learning rate $10^{-6}$).

### 4.4 Attention-Enhanced CNN with CBAM

To improve the ability of the CNN to selectively focus on disease-relevant spatial and channel features, the custom architecture was enhanced with the Convolutional Block Attention Module (CBAM). CBAM applies sequential channel and spatial attention operations, formally expressed as:

$$F' = M_C(F) \otimes F \quad (4)$$
$$F'' = M_S(F') \otimes F' \quad (5)$$

where F ∈ ℝ^(C×H×W) is the input feature map, M_C ∈ ℝ^(C×1×1) is the channel attention map, M_S ∈ ℝ^(1×H×W) is the spatial attention map, and ⊗ denotes element-wise multiplication.

The Channel Attention module computes attention weights by squeezing spatial dimensions through both average pooling and max pooling, passing each through a shared two-layer MLP with reduction ratio r = 16, and combining the outputs via element-wise addition followed by sigmoid activation:

$$M_C(F) = \sigma(MLP(AvgPool(F)) + MLP(MaxPool(F))) \quad (6)$$

The Spatial Attention module concatenates average-pooled and max-pooled channel descriptors along the channel axis and applies a 7×7 convolution followed by sigmoid activation:

$$M_S(F) = \sigma(f^{7\times7}([AvgPool(F); MaxPool(F)])) \quad (7)$$

The AttentionCNN architecture consists of four convolutional blocks (32, 64, 128, 256 filters), with CBAM modules inserted after blocks 3 and 4. Global Average Pooling is applied before the classifier. The model was trained using the PyTorch framework with the Adam optimizer ($\eta = 1 \times 10^{-4}$), weighted cross-entropy loss (Eq. 3), and a ReduceLROnPlateau scheduler (patience = 2, factor = 0.5). Early stopping (patience = 5) was applied and training was capped at 50 epochs.

### 4.5 Explainability and Generalization

**Gradient-weighted Class Activation Mapping (Grad-CAM)**

To provide visual interpretability of model predictions, Grad-CAM was applied to the best-performing AttentionCNN model. Grad-CAM computes a class-discriminative localization map by weighting the feature activation maps of the final convolutional layer (model.block4) with the gradient of the class score:

$$\alpha_k^c = (1/Z) \Sigma_i \Sigma_j (\partial y^c / \partial A_k^{ij}) \quad (8)$$

$$L_{Grad\text{-}CAM}^c = ReLU(\Sigma_k \alpha_k^c \cdot A_k) \quad (9)$$

where y^c is the score for class c, A_k is the k-th feature map of the target layer, and Z is the spatial normalization factor. The resulting heatmap L was upsampled to the original image size using bilinear interpolation and overlaid on the input image using the JET colormap. Grad-CAM was applied to a curated set of 10 test samples (8 correct predictions and 2 misclassifications) to provide qualitative insight into model behavior.

**Generalization Study**

To evaluate the cross-dataset robustness of the trained AttentionCNN + CBAM model, the architecture was independently re-trained on a second tomato leaf disease dataset (Kaggle: kaustubhb999/tomatoleaf). Class names were harmonized between datasets using a predefined label mapping. The same data split ratios (70/15/15), transformations, optimizer settings, and early stopping criteria were applied. Performance metrics on the second dataset's test split were compared against results obtained on the PlantVillage test split.

In addition to classification metrics, training time and inference (testing) time were recorded in seconds to assess computational efficiency. Confusion matrices and per-class ROC curves were generated for all models. All experiments were executed on an NVIDIA GPU-enabled Kaggle kernel. Random seeds were fixed at 42 across all libraries (Python random, NumPy, PyTorch, TensorFlow) to ensure reproducibility.

# 5.Results and Discussion

## Performance Metrics: Model Evaluation & Comparison

## Supervised Pre-trained Baselines:

We evaluated three deep learning models - ResNet50, VGG16, and EfficientNetB0 on a 10-class tomato disease classification dataset. The models were trained for up to 50 epochs with early

stopping, and metrics including accuracy, precision, recall, F1-score, AUC, training time, and testing time were recorded.

## Summary of Results:

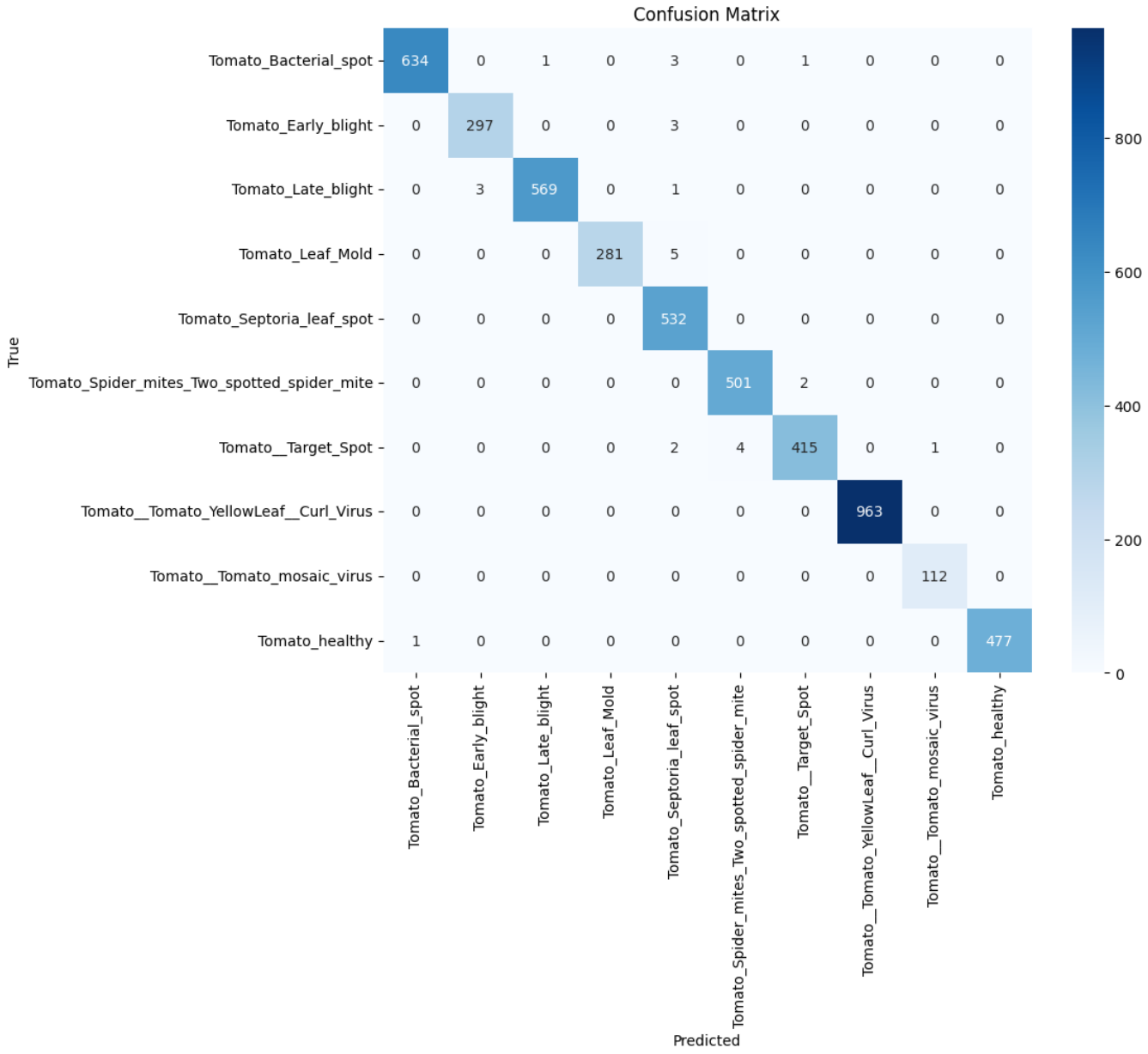
| Model | Accuracy | Precision | Recall | F1-score | AUC | Training Time | Testing Time |
|---|---|---|---|---|---|---|---|
| **ResNet50** | 99.44% | 99.37% | 99.35% | 99.36% | 1.000 | ~42.5 min | 17.22 sec |
| **VGG16** | 95.47% | 95.01% | 95.16% | 94.85% | 0.9992 | ~90 min | 28.70 sec |
| **EfficientNetB0** | 99.60% | 99.63% | 99.53% | 99.58% | 1.000 | ~29 min | 9.29 sec |

Fig 1: Table 1

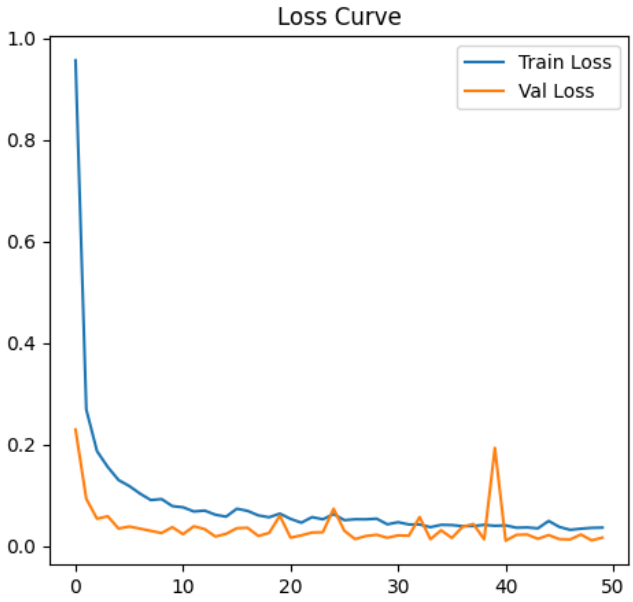
**Interpretation:** As shown in Table 1, the results indicate that EfficientNetB0 outperforms the other models, achieving the highest accuracy (99.60%), F1-score (99.58%), and lowest training (~29 min) and testing time (9.29 s), while maintaining an AUC of 1.000. ResNet50 also shows strong performance with comparable accuracy (99.44%) but higher computational cost. In contrast, VGG16 demonstrates lower accuracy (95.47%) and the highest training time (~90 min). Overall, EfficientNet50 offers the best balance between performance and efficiency for tomato disease classification.

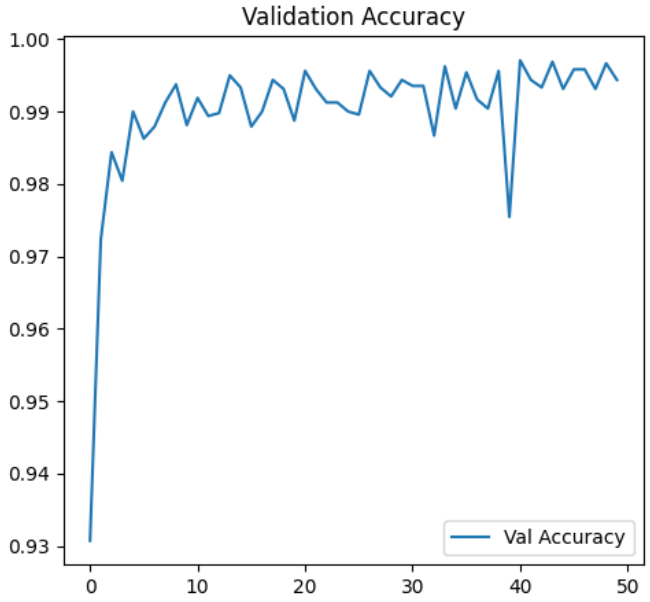
# ResNet50

## Confusion Matrix:

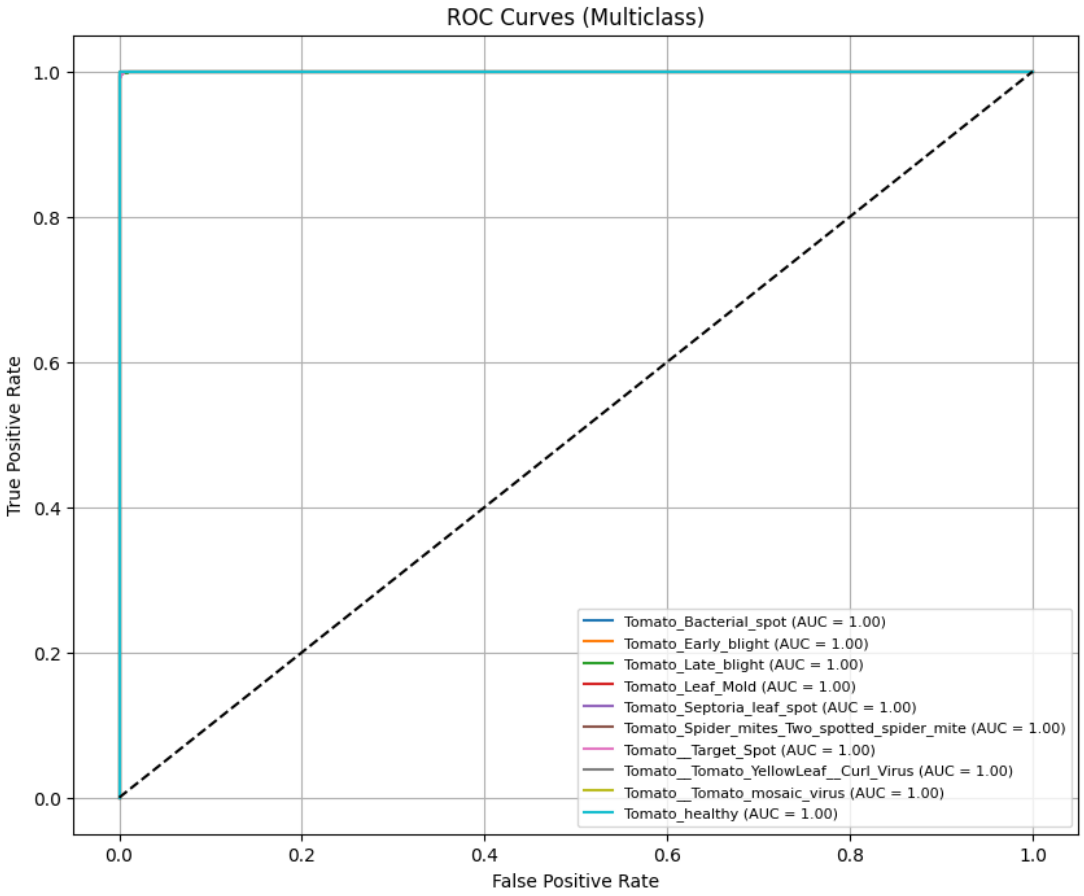


## Loss Curve and Validation Accuracy:



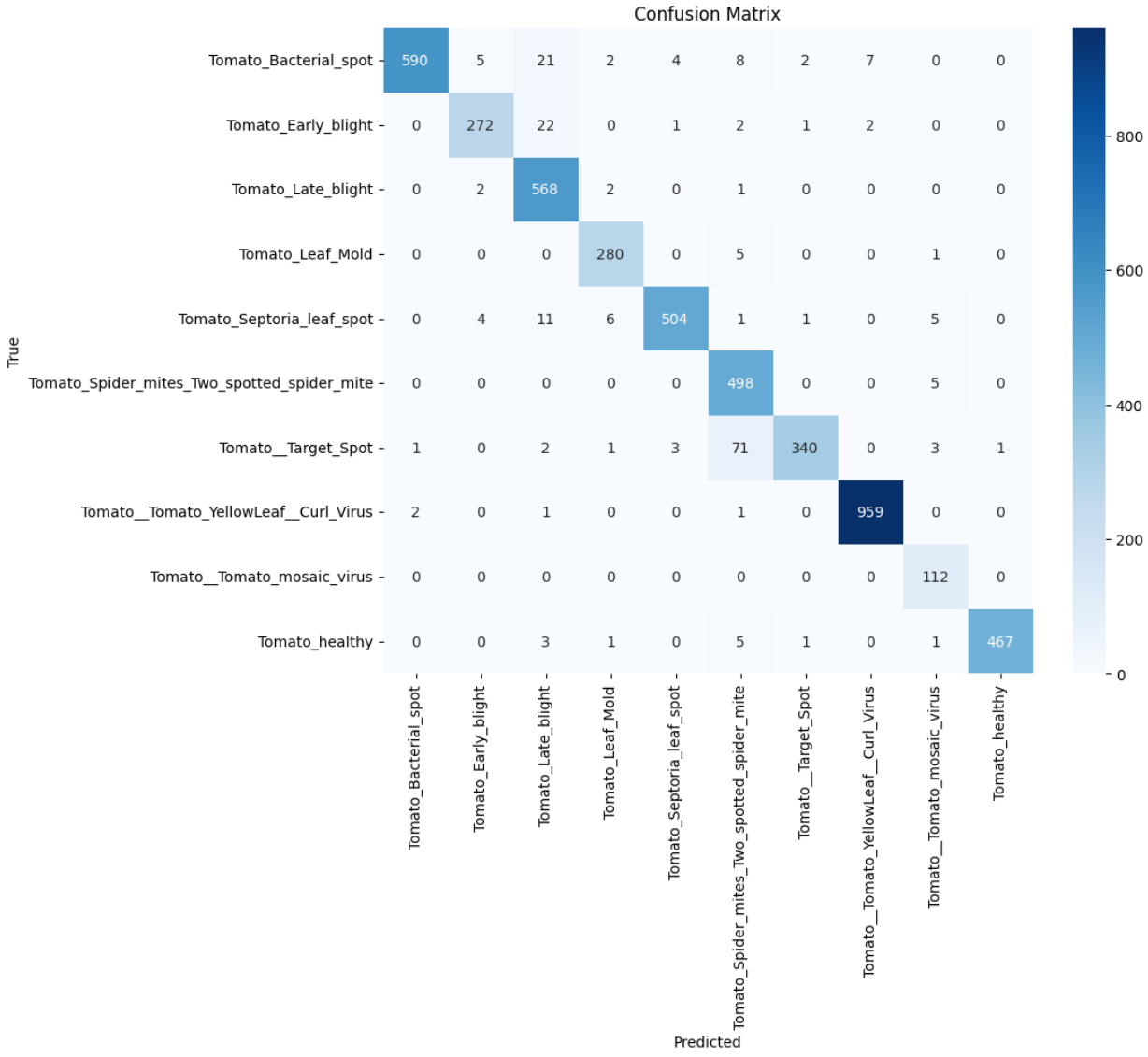




## ROC:

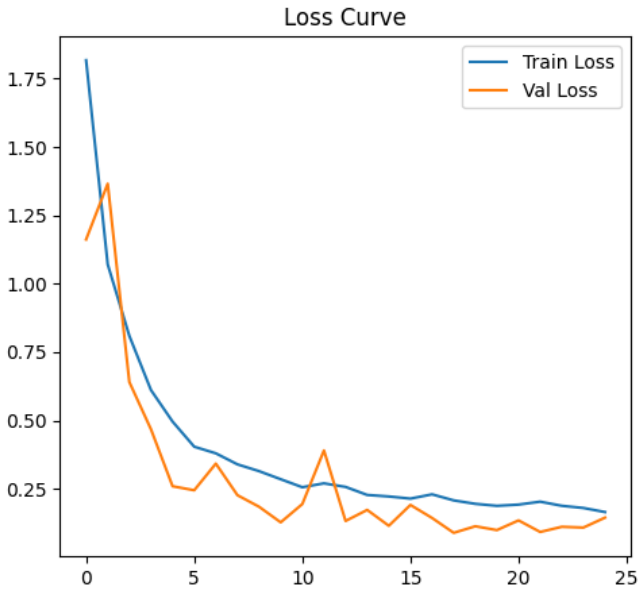


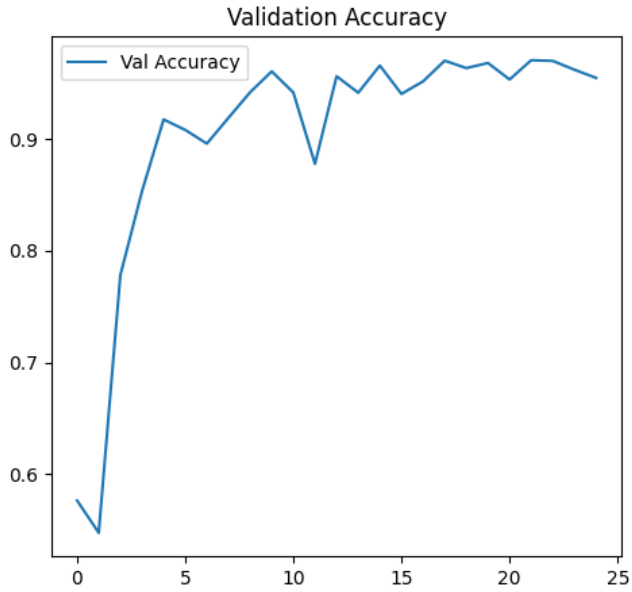
# VGG16

## Confusion Matrix:

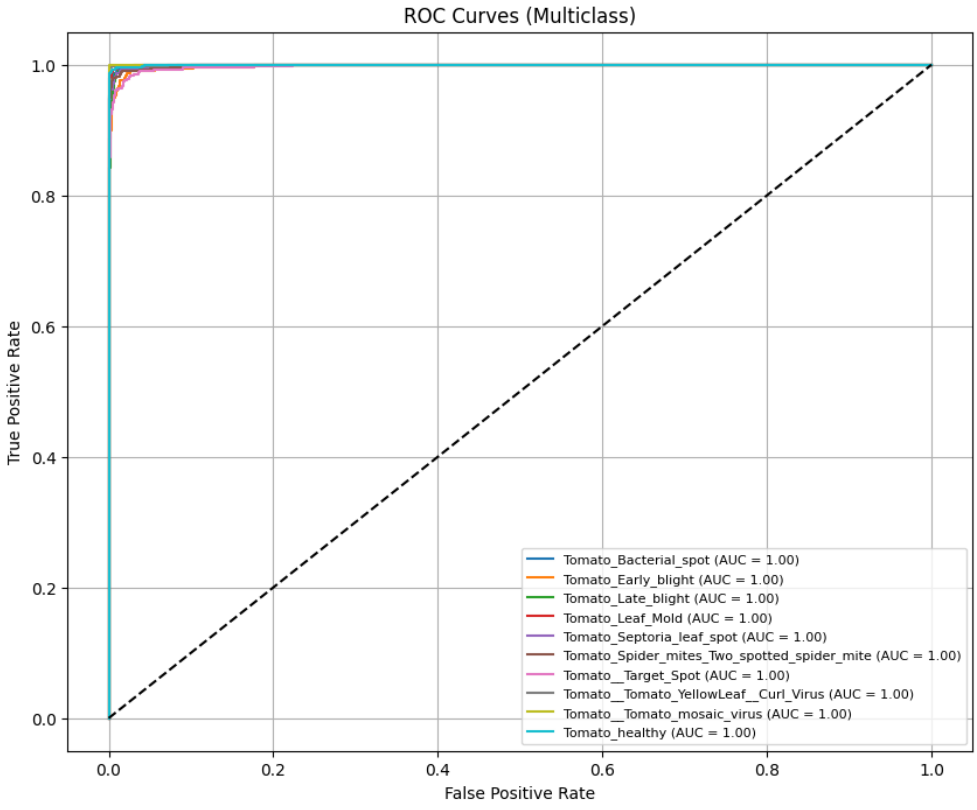


## Loss Curve and Validation Accuracy:



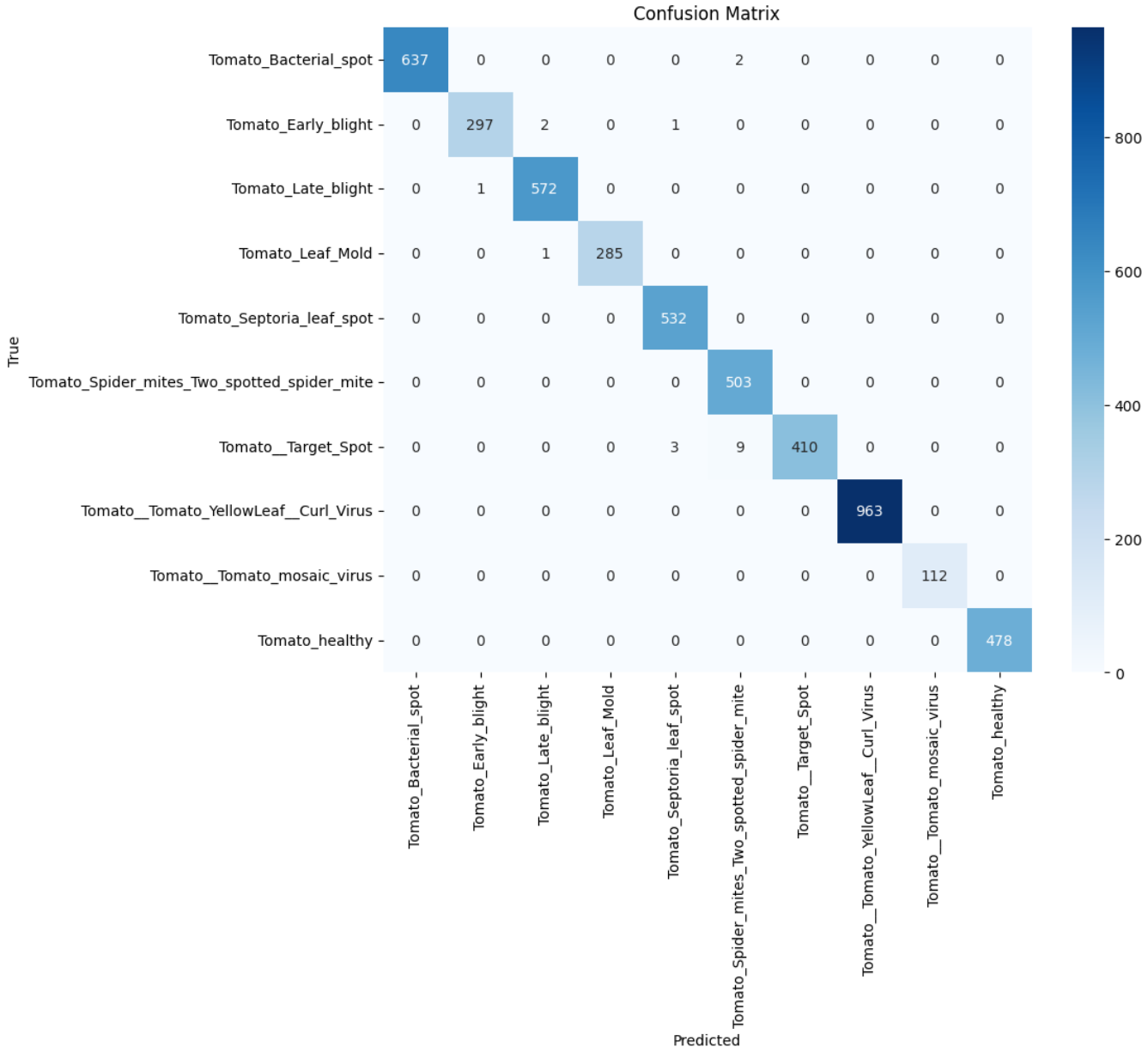




## ROC:

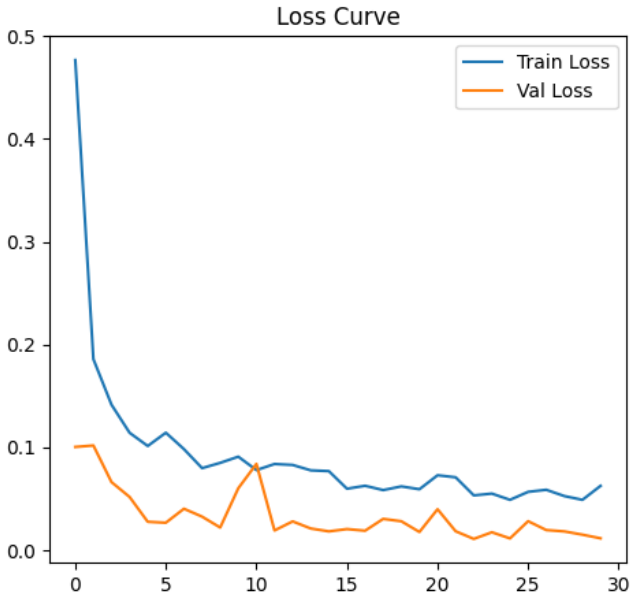


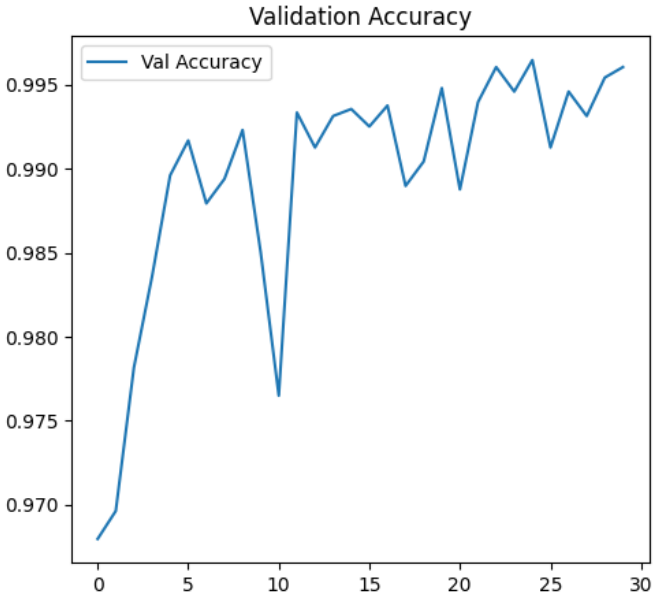
# EfficientNetB0

## Confusion Matrix:

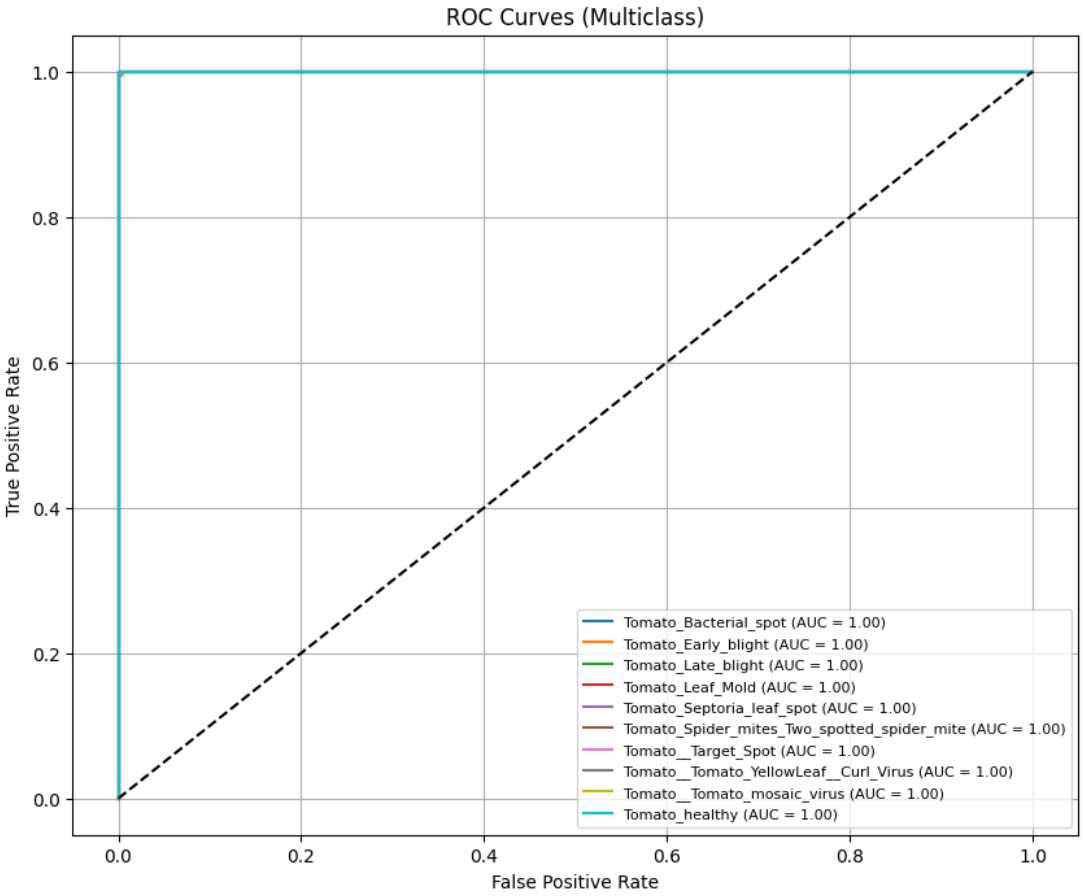


## Loss Curve and Validation Accuracy:




## ROC:



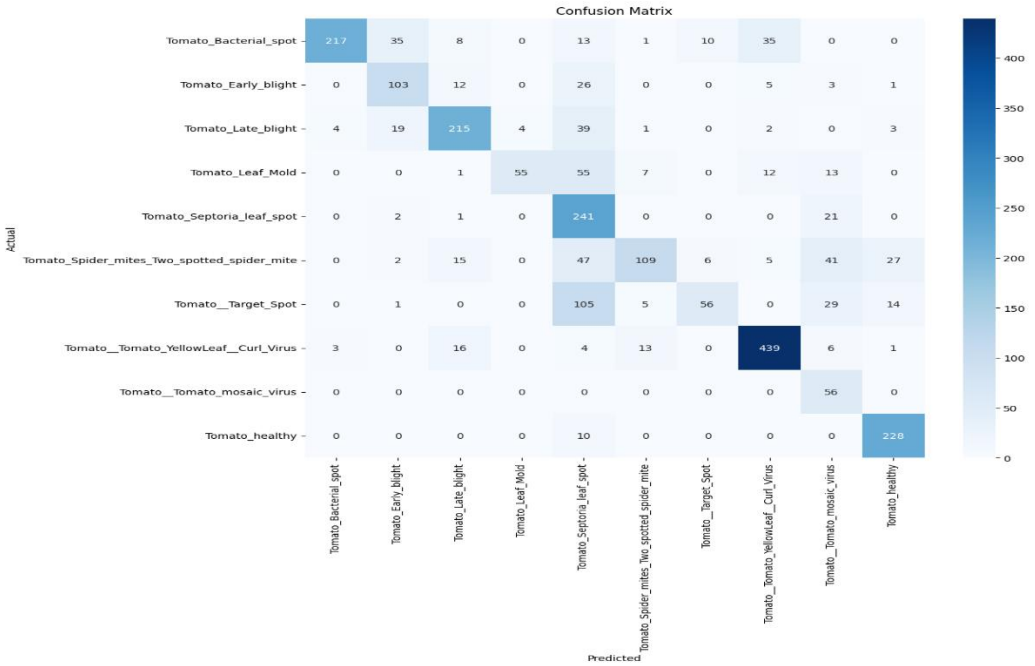
## Custom CNN Model:

We evaluated a custom Convolutional Neural Network (CNN) model on a 10-class tomato leaf disease classification dataset. The model was designed from scratch using multiple convolutional and pooling layers, along with batch normalization and ReLU activations to improve feature learning.
To enhance generalization and reduce overfitting, data augmentation techniques and class weighted training were applied. The model was trained for a maximum of 50 epochs using early stopping (patience = 5), which automatically terminated training when validation loss stopped improving. A separate train/validation/test split was used to ensure fair evaluation.
The model demonstrated stable learning behavior, although some class-wise imbalance and interclass similarity affected performance.

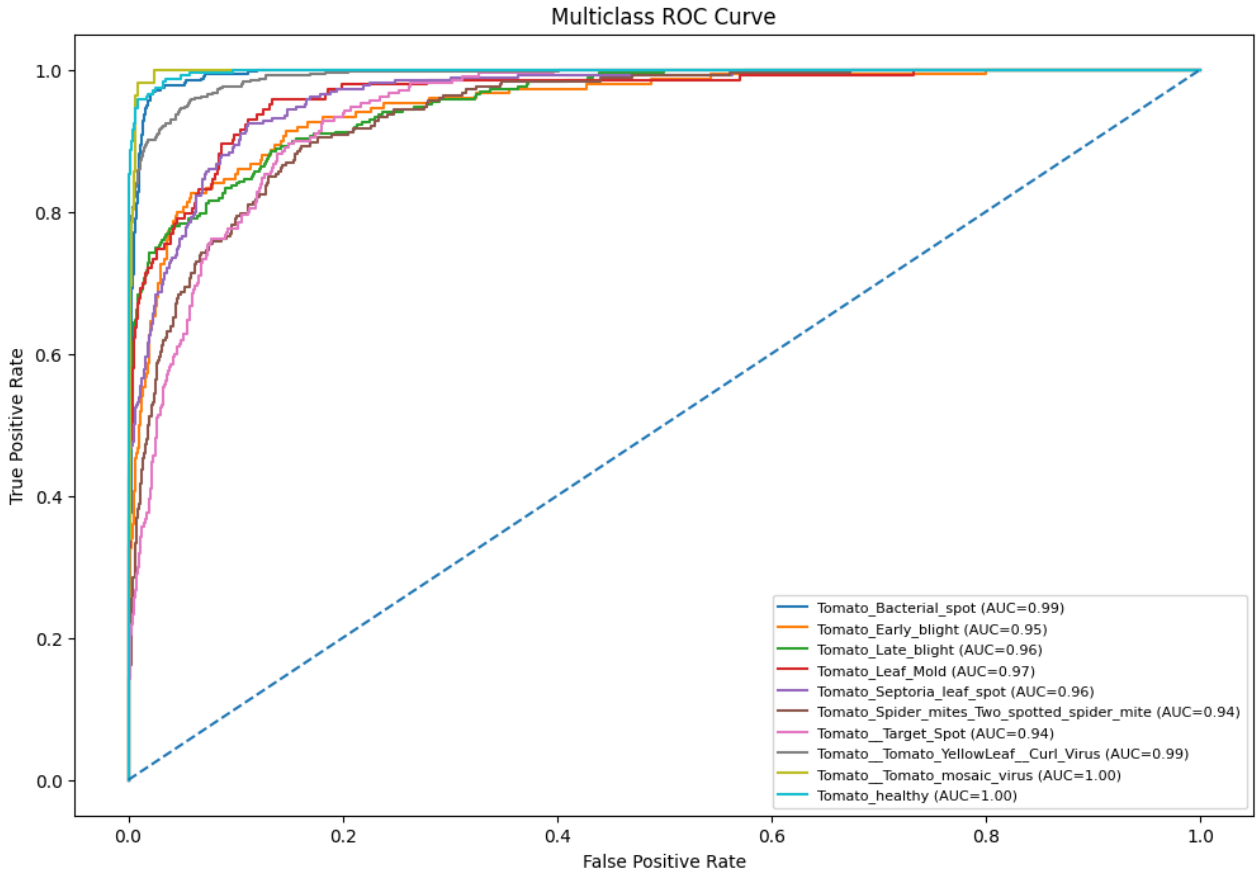
## Summary of Results:

| Model | Accuracy | Precision | Recall | F1-score | AUC | Training Time | Testing Time |
|---|---|---|---|---|---|---|---|
| **Custom CNN** | 71.56% | 74.09% | 69.78% | 66.20% | 0.9696 | 21.6 min | 11.27 sec |

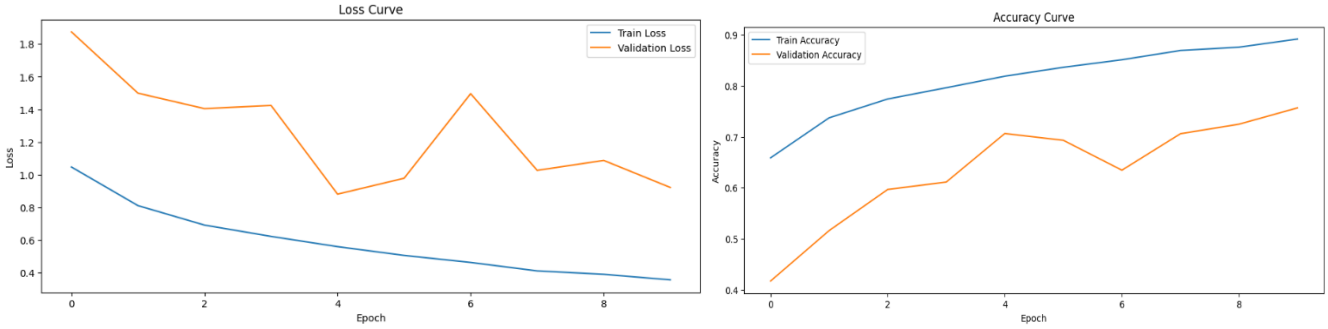
## Confusion Matrix:



## ROC Curve:




## Loss and Acuuracy Curve:



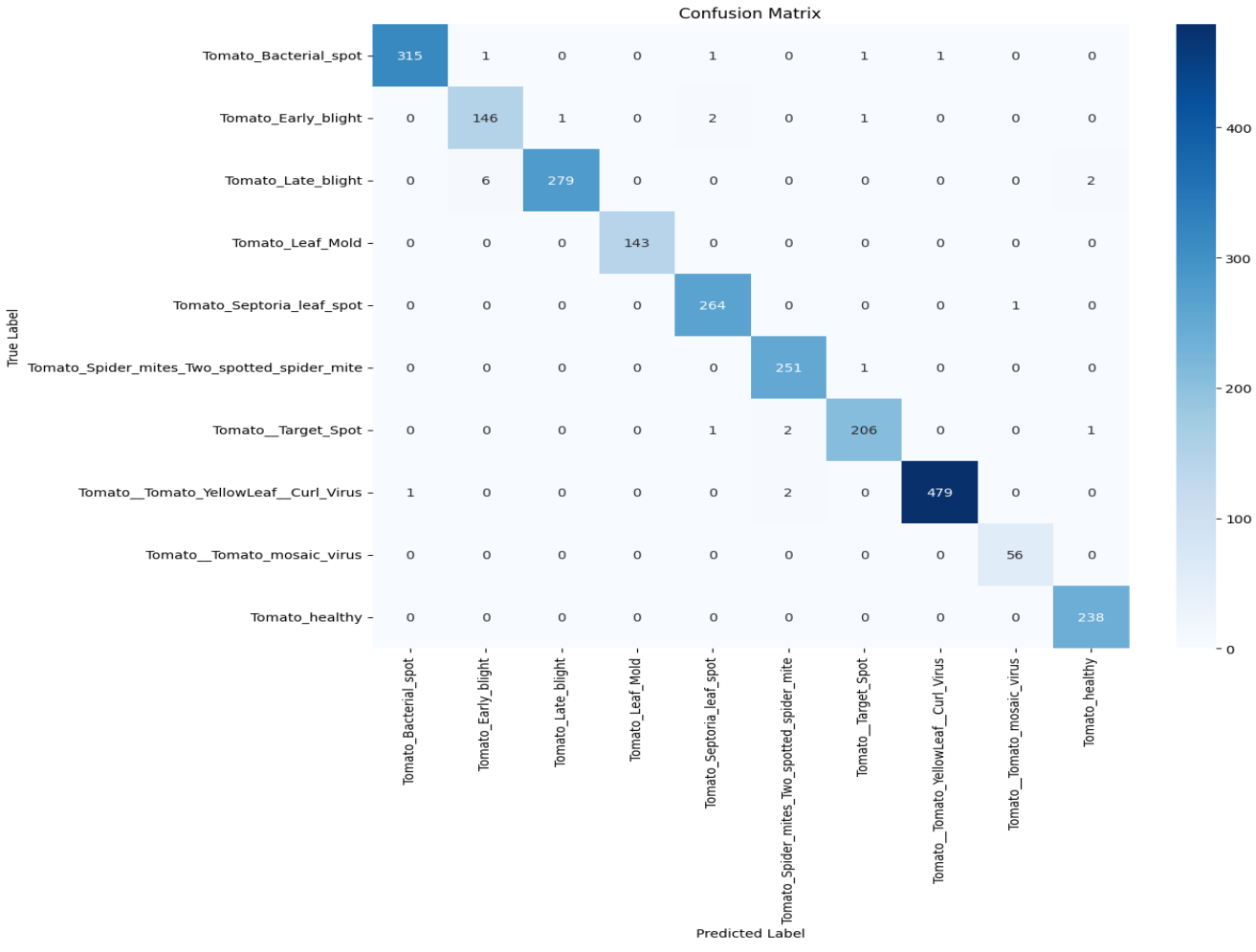
## Attention-Enhanced CNN Model:

In Task 4, the custom CNN model from Task 3 was enhanced by integrating a Convolutional Block Attention Module (CBAM) to improve feature extraction. The attention mechanism enables the network to focus on the most informative channel and spatial regions of the input image, which is particularly beneficial for identifying disease-specific patterns on plant leaves.
The CBAM module was applied after deeper convolutional layers to refine feature maps by emphasizing relevant features and suppressing irrelevant background information. The model was trained using the same preprocessing steps, data augmentation strategies, and hyperparameters as in Task 3 to ensure a fair comparison.
Class imbalance was handled using weighted loss, and early stopping was applied to prevent overfitting. The model was evaluated using a separate test set with comprehensive performance metrics.

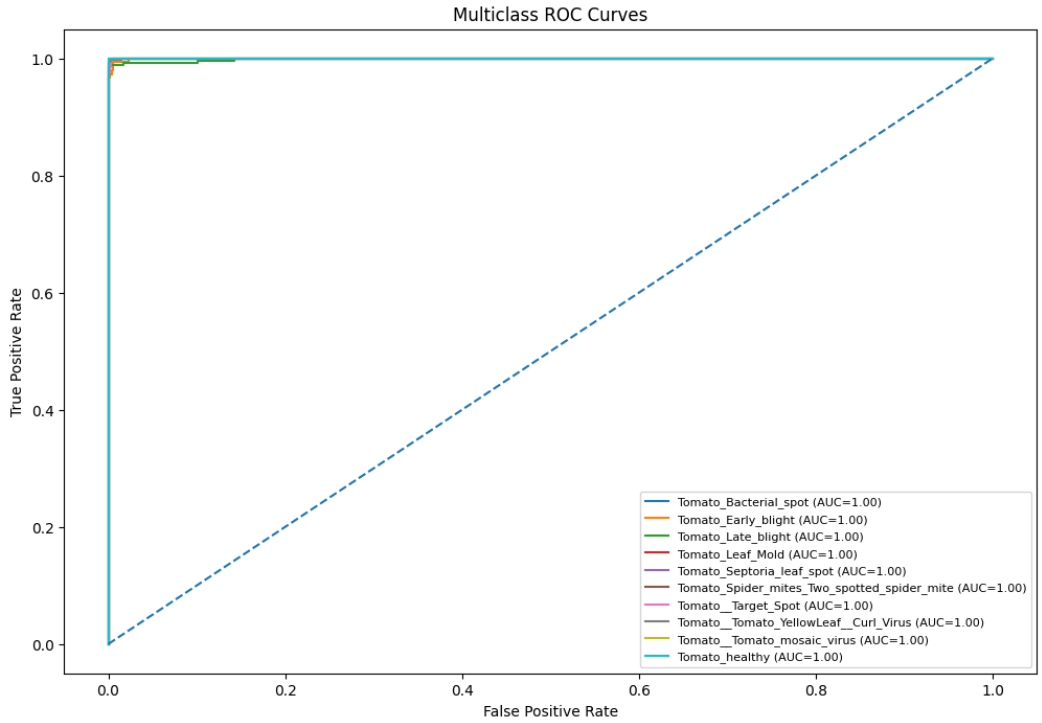
## Summary of Results:

| Model | Accuracy | Precision | Recall | F1-score | AUC | Training Time | Testing Time |
|---|---|---|---|---|---|---|---|
| **Attention-Enhanced CNN** | 98.95% | 98.7% | 98.99% | 98.84% | 0.9998 | 37.76 min | 15.583sec |

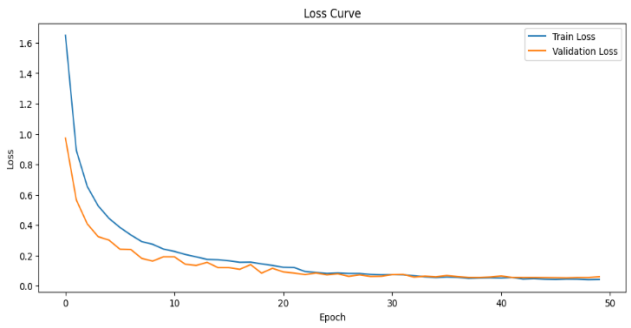
## Confusion Matrix:

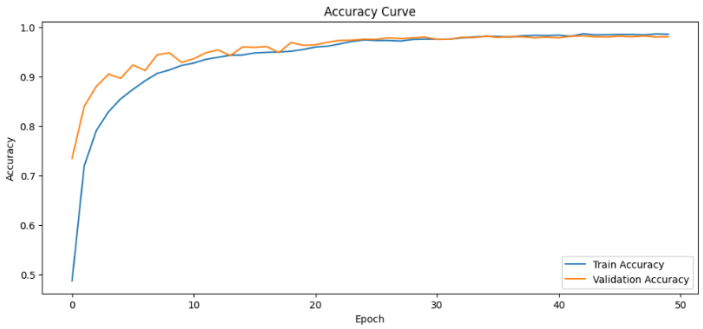


## ROC Curve:



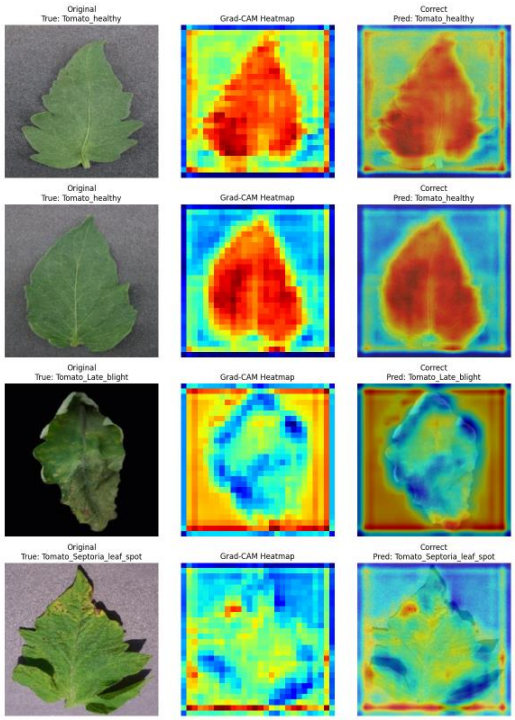

## Loss and Acuuracy Curve:

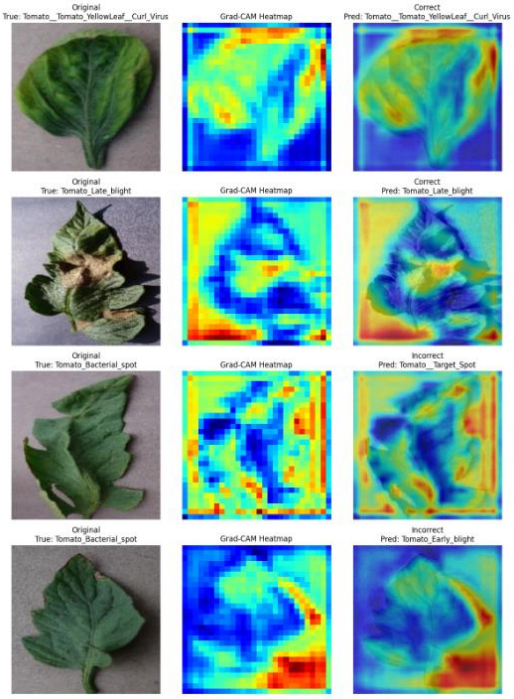


## Explainability using Grad-CAM

To interpret the decision-making process of the best-performing model, Gradient-weighted Class Activation Mapping (Grad-CAM) was applied to the AttentionCNN + CBAM model. Gradients were extracted from the final convolutional layer (block4) to generate class-discriminative heatmaps.
The test set contained 2,402 samples, of which 2,377 were correctly classified and only 25 were misclassified, yielding a 98.96% test accuracy. For visualization, 10 representative samples were selected — 8 correct predictions and 2 incorrect predictions — to provide qualitative insight into both successful and failure cases. An additional analysis was performed on the minority class, Tomato Mosaic Virus, to verify model behavior on underrepresented categories.
Results show that for correct predictions, the model consistently focuses on disease-relevant regions such as lesion spots, discolored patches, and infected tissue. For incorrect predictions, the heatmaps reveal that attention shifts to irrelevant background regions, explaining the misclassification. This confirms that the CBAM attention module effectively guides the model toward biologically meaningful features rather than dataset artifacts.




## Generalizability Testing

| Dataset | Accuracy | F1-score | AUC |
|---|---|---|---|
| **PlantVillage (Original)** | 98.96% | 0.9885 | 0.9998 |
| **Second Dataset (Unseen)** | 97.47% | 0.9746 | 0.9993 |

Fig 2: Table 2

**Interpretation**: The results in Table 2 show a minor accuracy drop of approximately 1.49%. This slight degradation is expected due to minor domain shifts in background variation and lighting differences. Nonetheless, retaining an accuracy of 97.47% on unseen external data indicates an exceptionally strong generalization capability.

# 6.Conclusion

This project successfully demonstrates the effectiveness of deep learning for automated tomato leaf disease classification. Through rigorous experimentation, we showed that while massive pretrained models like EfficientNetB0 provide state-of-the-art accuracy, carefully designed custom architectures equipped with spatial and channel attention mechanisms (CBAM) can achieve highly competitive results (98.96% accuracy). Furthermore, the application of Grad-CAM provided essential transparency, proving that the models successfully isolate relevant pathological features rather than relying on dataset artifacts. Finally, cross-dataset evaluation confirmed that the proposed approach is robust against minor domain shifts. The integration of attention mechanisms with explainability tools offers a powerful balance between high performance and diagnostic transparency, making it highly suitable for real-world agricultural deployment.

# References


[1] A. K. Pandey, D. Jain, T. K. Gautam, J. S. Kushwah, S. Shrivastava, R. Sharma, and P. Vats,"Tomato Leaf Disease Detection using Generative Adversarial Network-based ResNet50V2," Engineering Letters, vol. 32, no. 5, pp. 965–973, May 2024.

[2] Nagamani H. S., Sarojadevi H., "Tomato Leaf Disease Detection using Deep Learning Techniques," International Journal of Advanced Computer Science and Applications (IJACSA), Vol. 13, No. 1, 2022.

[3] Agarwal, M., Singh, A., Arjaria, S., Sinha, A., & Gupta, S. (2020). *ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network*. Procedia Computer Science, 167, 293–301. Elsevier.

[4] Xia Yuting. *Research on Tomato Leaf Disease Classification Based on Machine Learning*. Proceedings of CIBDA 2025, ACM, DOI: 10.1145/3746709.3746883

[5] Prama, T. T., & Oni, M. K. *Optimized Custom CNN for Real-Time Tomato Leaf Disease Detection*. Springer, ICCS 2025. DOI: 10.1007/978-3-031-97564-6_6